

Conceptos básicos de Git y GitHub

Tarea de Microcontroladores y Microprocesadores

1. ¿Cuál es la utilidad principal de Git?

Git es un sistema de control de versiones que permite registrar y administrar los cambios realizados en los archivos de un proyecto. Su utilidad principal es facilitar el seguimiento de las diferentes versiones del código, permitiendo saber qué cambios se realizaron, cuándo se hicieron y, si es necesario, regresar a una versión anterior. También facilita el trabajo colaborativo entre varias personas.

2. ¿Qué es un commit?

Un commit es un registro de los cambios realizados en un proyecto en un momento determinado. Cuando se realiza un commit, Git guarda una especie de fotografía del estado actual de los archivos que fueron incluidos. Normalmente, cada commit lleva un mensaje descriptivo que permite identificar qué se modificó, por ejemplo: "Corrección del programa de lectura del sensor". Los commits permiten mantener un historial ordenado del desarrollo del proyecto.

3. ¿Qué es un branch?

Un branch (rama) es una línea independiente de desarrollo dentro de un repositorio de Git. Permite realizar cambios o desarrollar nuevas funciones sin modificar directamente la versión principal del proyecto. Por ejemplo, se puede crear un branch llamado "prueba-sensor" para desarrollar una función relacionada con un sensor mientras el branch principal permanece estable. Una vez terminados y comprobados los cambios, estos pueden integrarse nuevamente al branch principal.

4. ¿Qué es un Pull Request en GitHub?

Un Pull Request (PR) es una solicitud realizada en GitHub para integrar los cambios de un branch con otro branch, normalmente con el branch principal. Permite que los integrantes de un equipo revisen el código antes de incorporarlo al proyecto principal. Durante el Pull Request se pueden realizar comentarios, solicitar modificaciones y comprobar que los cambios funcionen correctamente. Cuando la revisión termina y se aprueba, los cambios pueden combinarse mediante un proceso llamado merge.

5. ¿Cómo actualizar el repositorio contra el branch master?

Para actualizar el repositorio local con los cambios que existen en el branch master del repositorio remoto, primero se debe cambiar al branch correspondiente y después descargar e integrar los cambios. Por ejemplo, si el branch principal se llama master, se puede utilizar:

```
git checkout master  
git pull origin master
```

El primer comando cambia al branch master y el segundo obtiene los cambios del repositorio remoto (origin) y los integra en la copia local. En repositorios modernos, el branch principal suele llamarse “main” en lugar de “master”, por lo que en ese caso se utilizaría:

```
git checkout main  
git pull origin main
```

6. ¿Bajo que condiciones una herramienta como Git necesita apoyo de un humano para saber como integrar cambios locales con cambios remotos?

Git puede fusionar automáticamente casi siempre que los cambios estén en archivos o líneas distintas. Pero cuando dos ramas modifican la misma línea o el mismo bloque de un archivo (o una rama borra un archivo que otra modificó), Git no puede decidir cuál versión conservar y se detiene: marca el archivo con conflicto y te pide que intervengas, indicando la situación con una "bandera roja". En ese momento el desarrollador debe abrir el archivo, revisar las marcas de conflicto (<<<<<<, =====, >>>>>>), decidir qué contenido conservar, y luego hacer git add + git commit para cerrar la fusión.

7. ¿Qué es una Prueba Unitaria o Unittest en el contexto de desarrollo de software?

Una prueba unitaria o unittest es una prueba automatizada que verifica el comportamiento de la unidad más pequeña posible de código, normalmente una función, método o clase individual, de forma aislada del resto del sistema. Consiste en darle a esa unidad entradas controladas y conocidas de antemano, y comparar la salida que produce contra la salida que se espera obtener; si coinciden, la prueba pasa, y si no coinciden, la prueba falla, lo cual indica que existe un error en esa parte específica del código. Estas pruebas suelen escribirse con la ayuda de un framework, como unittest de la librería estándar de Python o pytest, que permite definir muchas pruebas pequeñas y ejecutarlas todas de manera automática y repetible.

Las pruebas unitarias son importantes porque permiten detectar errores de forma temprana, justo en el lugar donde ocurren, y además funcionan como una red de seguridad: si en el futuro se modifica el código y algo se rompe, las pruebas fallan de inmediato y alertan al desarrollador, evitando que el error pase desapercibido (Python Software Foundation, 2026).

8. Bajo el contexto de pytest. ¿Cuál es la utilidad de un “assert”?

assert es la instrucción que declara la condición que debe cumplirse para que la prueba se considere exitosa. Si la expresión evaluada es verdadera, pytest continúa sin problema; si es falsa, pytest marca esa prueba como fallida y muestra automáticamente un resumen de qué valores se compararon, sin necesidad de escribir mensajes de error manuales (a diferencia de assertEquals en unittest) (pytest-dev team, 2026).

9. ¿Qué es Flake 8?

Es una herramienta de análisis estático ("linter") para Python que combina tres herramientas en una: pyflakes (detecta errores lógicos como variables no usadas o imports faltantes), pycodestyle (verifica que el código siga la guía de estilo PEP 8) y mccabe (mide la complejidad ciclomática del código). Se ejecuta desde la terminal sobre un archivo o carpeta y reporta cada línea con problemas.

10. Comente sobre la utilidad de la aleatorización en pruebas de código.

Usar valores aleatorios dentro de las pruebas (en lugar de valores fijos) ayuda a que la prueba no dependa de un único caso particular y así se reduzca la posibilidad de que el código "pase por casualidad" con un valor específico, mientras falla con otros. En pruebas de software, permite automatizar pruebas y detectar errores sin necesidad de conocer el flujo interno del programa, aunque suele requerir muchas repeticiones para tener buena cobertura de casos.

Referencias:

Python Software Foundation. (2026). unittest — Unit testing framework. Python 3 documentation. <https://docs.python.org/3/library/unittest.html>

pytest-dev team. (2026). Get started. pytest documentation. <https://docs.pytest.org/en/stable/getting-started.html>